UA-GNN correction plan: updated implementation status

Review date: 2026-10-10. Repository: saisidd007/UA_GNN_Traffic_Forecasting. This status preface supersedes the older status pages; the supplied plan follows unchanged.

The uploaded LaTeX manuscript was checked against repository code and raw benchmark data. The current corrected small model and the separate manuscript interpretation are new code paths. Neither reproduces the manuscript's reported 1.67/3.72 mph PEMS-BAY or 3.46/6.57 mph METR-LA claims.

Measured 70/10/20 corrected-small-model runs

Dataset	MAE / RMSE (mph)	95% coverage	Best epoch	Epochs done / max
METR-LA	4.079 / 7.717	96.6%	49	50 / 50
PEMS-BAY	2.074 / 4.499	97.2%	44	50 / 50

METR-LA: persistence baseline 4.243/8.191 mph; mean 95% interval width 32.70 mph; 14,940,228 observed targets.

PEMS-BAY: persistence baseline 2.170/5.125 mph; mean 95% interval width 14.34 mph; 40,608,096 observed targets.

Both use CPU, seed 42, batch size 32, 8,192 sampled training windows per epoch, full validation/test windows, and 10 MC passes. Checkpoint selection uses validation MSE only. These are single-seed runs of a smaller model. They are not a reproduction of the paper architecture.

The official METR-LA DCRNN adjacency was obtained and its sensor order verified; it has 1,515 directed non-self edges. The earlier 70/15/15 METR-LA result used a geographic proxy. PEMS-BAY has 2,369 non-self edges at the same threshold, contrary to the manuscript's apparent 1,515 edge claim for that dataset.

Previously reported 70/15/15 small-model results remain archival: METR-LA 4.043/7.502 mph with proxy graph; PEMS-BAY 2.135/4.572 mph. Their Gaussian 95% interval coverage was 96.7% and 97.5%, respectively. ECE across nominal 10–95% levels was 0.175 and 0.164; high coverage alone did not establish calibration.

METR-LA reduced-budget five-seed check: MAE 4.366 ± 0.109 mph (1024 train windows/epoch, 512 sampled test windows, 12 epochs maximum); matched persistence 4.221 mph. All seeds selected the final epoch, so this undertrained variability check is not a final five-seed benchmark.

PEMS-BAY reduced-budget five-seed check: MAE 2.302 ± 0.015 mph (1024 train windows/epoch, 512 sampled test windows, 12 epochs maximum); matched persistence 2.187 mph. All seeds selected the final epoch, so this undertrained variability check is not a final five-seed benchmark.

METR-LA MC stability on 64 fixed windows: 10 vs 50 passes gave MAE 4.008 vs 4.003 mph, 95% coverage 96.87% vs 96.88%.

PEMS-BAY MC stability on 64 fixed windows: 10 vs 50 passes gave MAE 2.068 vs 2.064 mph, 95% coverage 97.34% vs 97.39%.

Limited-budget architecture pilots (preliminary)

PEMS-BAY manuscript interpretation: 3.881/7.617 mph, persistence 2.187/5.184 mph; 256 train windows/epoch, 512 sampled test windows, best epoch 20.

PEMS-BAY without diffusion: 4.293/7.965 mph, persistence 2.187/5.184 mph; 256 train windows/epoch, 512 sampled test windows, best epoch 19.

METR-LA manuscript interpretation: 6.491/12.575 mph, persistence 4.221/8.137 mph; 256 train windows/epoch, 512 sampled test windows, best epoch 19.

Matched PEMS-BAY pilot ablation: removing diffusion increased MAE by 0.413 mph (10.6%). A descriptive 14-window block bootstrap gives [0.353, 0.476] mph. Targets and masks match exactly; the single seed and sampled training budget cannot validate the manuscript's 23.617 normalized-MAE claim.

METR-LA small model, deterministic inference on 6844 test windows: MAE 4.071 mph, 95% coverage 96.6%, mean width 32.46 mph. This reuses the trained checkpoint.

PEMS-BAY small model, deterministic inference on 10413 test windows: MAE 2.067 mph, 95% coverage 97.1%, mean width 14.12 mph. This reuses the trained checkpoint.

PEMS-BAY architecture pilot, deterministic inference on 512 test windows: MAE 3.879 mph, 95% coverage 95.4%, mean width 22.14 mph. This reuses the trained checkpoint.

METR-LA architecture pilot, deterministic inference on 512 test windows: MAE 6.479 mph, 95% coverage 96.2%, mean width 44.72 mph. This reuses the trained checkpoint.

METR-LA small model CPU inference: 14,520 active parameters, median 39.7 ms per batch of 16 with one pass; peak process RSS 385 MiB (20 measured batches, 1 thread).

PEMS-BAY small model CPU inference: 14,520 active parameters, median 60.4 ms per batch of 16 with one pass; peak process RSS 497 MiB (20 measured batches, 1 thread).

PEMS-BAY architecture pilot CPU inference: 222,480 active parameters, median 3529.1 ms per batch of 16 with one pass; peak process RSS 858 MiB (10 measured batches, 1 thread).

PEMS-BAY no-diffusion pilot CPU inference: 140,304 active parameters, median 1959.3 ms per batch of 16 with one pass; peak process RSS 854 MiB (10 measured batches, 1 thread).

METR-LA architecture pilot CPU inference: 222,480 active parameters, median 2268.4 ms per batch of 16 with one pass; peak process RSS 698 MiB (10 measured batches, 1 thread).

METR-LA small model synthetic failures, 128 sampled windows, 3 MC passes: random 50% MAE 5.09 mph (1.24× clean), 95% coverage 95.6% and width 39.4 mph; clustered 50% MAE 5.48 mph (1.33× clean), 95% coverage 93.4% and width 35.0 mph.

PEMS-BAY small model synthetic failures, 128 sampled windows, 3 MC passes: random 50% MAE 3.44 mph (1.59× clean), 95% coverage 90.2% and width 13.6 mph; clustered 50% MAE 3.61 mph (1.67× clean), 95% coverage 79.7% and width 10.2 mph.

PEMS-BAY architecture pilot synthetic failures, 16 sampled windows, 3 MC passes: random 50% MAE 3.85 mph (1.01× clean), 95% coverage 94.6% and width 21.0 mph; clustered 50% MAE 3.99 mph (1.05× clean), 95% coverage 93.4% and width 19.5 mph.

Status against the supplied correction plan

1. Data preprocessing — DONE in new paths; original loaders retained. Chronological complete-target splits, causal forward fill, train-only median/IQR fit, zero-target masking. 70/10/20 boundaries and missing counts are in MANUSCRIPT_AUDIT.md. Zero-as-missing is a documented benchmark convention, not proof for every reading.

2. Temporal architecture — PARTIAL. The corrected small model and separate four-block paper interpretation use the correct time and node axes. Unit tests and shape table pass. The paper omits layer details: the interpretation has 222,480 parameters, not its claimed approximately 304K.

3. Validation/test separation — DONE in new paths. Validation selects the best checkpoint; final test evaluation occurs after training. Legacy scripts remain historical artifacts.

4. MC dropout and normalization — DONE in new paths. Only Dropout modules are activated at inference; BatchNorm remains in evaluation mode in the paper interpretation.

5. Uncertainty/calibration — PARTIAL. Combined predictive variance, 50/80/90/95% coverage and widths, Gaussian CRPS, horizon checks, and reliability plots are implemented. No trained no-aleatoric-head ablation was run: removing its variance without a defined replacement distribution makes interval coverage/CRPS undefined. The saved-variance component check and no-MC inference check are narrower tests. There is no demonstrated calibration guarantee.

6. Graph/diffusion — PARTIAL. Both source adjacencies are audited; directed one/two-hop unit tests pass. The four-block model includes forward and reverse diffusion. An adequately trained multi-seed no-diffusion ablation is still needed to assess the paper's extreme 23.617 claim.

7. Core experiments — PARTIAL. The corrected small model is rerun on both datasets. Reduced-budget five-seed runs are exploratory; five adequately trained seeds and a validated paper-architecture reproduction remain outstanding.

8. Sensor failure — PARTIAL. Code supports matched random, persistent, clustered, and temporal-block failure at 10/20/30/40/50%, three corruption seeds, two causal input-fill strategies, a persistence reference, and MC interval diagnostics. Any measured robustness table must identify its model and test-window sample.

9. Additional metrics — PARTIAL. CRPS, horizon scores, latency and memory measurement are implemented; matched broad baseline reproductions are not available.

10. Two-student division — NOT APPLICABLE. This is staffing advice, not a technical correction.

11. Paper consistency — NOT COMPLETE. The manuscript's accuracy, ablation, robustness and calibration claims are not validated by these new results. See MANUSCRIPT_AUDIT.md for concrete corrections and unsupported statements.

Manuscript-specific decisions and remaining evidence

Implemented: the source-backed METR-LA graph; configurable manuscript split; four-block causal temporal/directed diffusion/attention/BiLSTM model interpretation; temporal embeddings; mean/positive-variance heads; selective MC dropout; synthetic failure evaluator; independent prediction audit; reproducible seed runner; CPU efficiency tool.

Rejected as stated: unrestricted linear interpolation, because it can use future measurements; the PEMS-BAY 1,515-edge count; the claim that the softplus derivative stays bounded away from zero; treating literature baseline scores as matched experiments; asserting that a backward LSTM state sees forecast future; claiming calibration or anticipatory uncertainty widening without testing it.

Unresolved: the manuscript does not fully specify embedding width, head topology, masking schedule, annealing direction or exact model definition. Thus the 304K parameters and manuscript point scores cannot be reconstructed from its prose alone. The current cloud machine has no GPU; all new measured runs use CPU. Limited training windows or epochs must remain labelled exploratory.

The original five-page correction plan begins on the next page. Its instructions are proposals to assess, not evidence that a specific code change is correct or that an old reported result is valid. MANUSCRIPT_REQUIRED_EDITS.md lists concrete replacements needed before revising the paper.

UA-GNN Traffic Forecasting

Experimental Correction and Validation Plan

Repository: https://github.com/Chakri745/UA_GNN_Traffic_Forecasting

Purpose: Student-facing instructions to correct the experimental pipeline, verify model behaviour, rerun core experiments and strengthen the paper's evidence.

Guiding rule: Preserve original code and results, record every change, and do not overwrite old outputs. Report corrected results even if less favourable.

Execution order

Correct data/model issues first; validate uncertainty and evaluation next; then run repeated-seed, ablation and robustness experiments. Do not spend the full experimental budget repeating a pipeline before checking its correctness.

1. Correct data preprocessing and eliminate leakage

Priority: Essential
Why: The loader appears to fit preprocessing before chronological splitting; backward filling may use later observations to fill earlier missing values.

Files to inspect: Dataset loader/preprocessing module, imputation functions and split creation.

1. Split raw time series chronologically into training, validation and test periods using the intended proportions.

2. Fit scaler and any learned imputer on training data only; apply the fitted transformations to validation/test without refitting.

3. Remove backward filling when it uses future observations unavailable at forecasting time. Use a documented causal alternative or an observation-mask-aware method.

4. Ensure sliding input windows and forecast targets do not cross boundaries in a way that exposes future targets to training.

5. Check how zero values and missing readings are represented; do not conflate genuine low speed with missingness.

Deliverables: Revised preprocessing code, split timestamps, missing-value counts, scaler-fit details and leakage-check report.

Acceptance test: No validation/test values fit preprocessing; no future observation imputes a past input; training examples contain no test-period targets.

2. Correct the temporal model architecture

Priority: Essential
Why: Temporal convolution and LSTM operations may act on the sensor/node dimension instead of the 12-step historical-time dimension. Confirm from tensor shapes before modifying code.

Files to inspect: Model definition containing temporal convolution, LSTM and spatial/diffusion operations.

1. Print and document tensor shape after every major operation, identifying batch, sensor, historical time-step and feature/channel dimensions.

2. Ensure temporal convolution operates along the 12 historical time steps, not across sensors.

3. Ensure the LSTM sequence axis represents historical time; document reshaping and permutation operations.

4. Ensure graph diffusion or graph attention operates along the sensor-graph dimension.

5. Verify output shape is batch × sensors × forecast horizon, with the intended uncertainty output.

Deliverables: Corrected code, tensor-shape table, matching model diagram and a successful forward-pass test.

Acceptance test: Operations match the mathematical formulation; do not merely reshape tensors to make code run.

3. Correct validation, testing and checkpoint selection

Priority: Essential
Why: The training loop appears to evaluate test performance repeatedly, risking decisions influenced by held-out test results.

Files to inspect: Training loop and evaluation/checkpoint-selection functions.

1. Use training data to update model parameters.

2. Use validation loss for checkpoint selection, hyperparameter tuning and early stopping.

3. Do not inspect test metrics every epoch; evaluate test only after the configuration and checkpoint procedure are finalised.

4. Keep test data untouched during hyperparameter selection.

5. Apply consistent split and evaluation rules to PEMS-BAY and METR-LA.

Deliverables: Training logs, selected checkpoint, validation curves and one final test report per dataset.

Acceptance test: Test performance does not determine early stopping, checkpoint selection or hyperparameter decisions.

4. Fix Monte Carlo dropout and batch normalisation

Priority: Essential
Why: Calling model.train() during MC inference can also put batch-normalisation layers into training mode.

Files to inspect: Model-evaluation and Monte Carlo sampling functions.

1. Put the model into evaluation mode before inference.

2. Enable dropout layers selectively for MC sampling while keeping batch-normalisation layers in evaluation mode.

3. Start with 30–50 stochastic passes for debugging and check stability as the pass count increases.

4. Confirm deterministic evaluation is reproducible and MC predictions vary when dropout is enabled.

5. Record the number of sampling passes and dropout configuration.

Deliverables: Tested inference function, a test demonstrating intended dropout behaviour and example stochastic predictions.

5. Correct uncertainty estimation and calibration

Priority: Essential
Why: The calibration script appears to use MC prediction spread alone for intervals, potentially omitting predicted aleatoric variance.

Files to inspect: Variance head, uncertainty loss, MC prediction function and calibration/evaluation script.

1. Let $\mu_m(x)$ be the predicted mean and $\sigma^2_m(x)$ the predicted aleatoric variance from MC pass m. Predictive mean is the average of $\mu_m(x)$.

2. Epistemic variance is the average squared deviation of $\mu_m(x)$ from the predictive mean.

3. Aleatoric variance is the average of $\sigma^2_m(x)$, after conversion from log variance if required.

4. Combine predictive variance as epistemic variance + aleatoric variance. Use a consistent variance convention and denominator.

5. For an approximately Gaussian distribution, a nominal 95% interval is predictive mean ± 1.96 × predictive standard deviation; test empirical coverage.

6. If outputs are normalised, inverse-transform predictions/intervals correctly. For nonlinear transformations, transforming interval endpoints and recomputing variance are not interchangeable.

Deliverables: MAE/RMSE; coverage at 50%, 80%, 90%, 95%; mean interval width; calibration plots; results by dataset/horizon; aleatoric, epistemic and total variance comparisons.

Acceptance test: Intervals used in evaluation follow the same uncertainty definition claimed in the manuscript.

6. Verify graph and diffusion implementation

Priority: Essential
Why: Graph direction, transition-matrix convention and diffusion order must match the equations and implementation.

Files to inspect: Graph construction, adjacency/transition matrix creation and diffusion component.

1. Define whether A_ij means an edge from sensor i to sensor j.

2. Verify row/column convention of transition matrices and feature aggregation.

3. Check whether the graph is directed or symmetrised; verify forward/reverse diffusion matrices.

4. Confirm whether the implementation uses one-hop propagation or multiple powers of the transition matrix.

5. Record node count, directed edge count, self-loops and distance threshold for each dataset.

6. Validate graph construction against the actual sensor adjacency files.

Deliverables: Graph statistics for both datasets, documented adjacency convention, a small hand-constructed graph unit test and equations consistent with code.

Special check: diffusion ablation

• The reported normalised MAE increase from 0.4392 to 23.617 when diffusion is removed is unusually large and must be verified.

• Use identical splits, scaling, seeds, training budget and evaluation code for full and no-diffusion variants.

• Check output scale/shape, exploding predictions, invalid values and inconsistent loss reductions.

• Recompute metrics independently from saved predictions and repeat across seeds.

• Do not discard the result simply because it is extreme; establish whether it is reproducible.

7. Re-run core experiments systematically

Priority: Essential after code fixes
Why: Old results are provisional if the pipeline or architecture changes.

Files to inspect: Experiment configuration, training scripts, checkpointing and metric scripts.

1. Run corrected model on PEMS-BAY and METR-LA.

2. Use at least five random seeds for final model results and key ablations where feasible; if fewer, report the exact number and treat results as preliminary.

3. Save each run's seed, configuration, best validation epoch, checkpoint, logs, predictions and metrics.

4. Report mean and standard deviation across runs and horizon-specific results.

5. Run final test evaluation only after selecting configuration using validation data.

Deliverables: Reproducible experiment folder and tables reporting mean ± standard deviation.

Core comparisons to retain

• Proposed UA-GNN.

• Proposed model without diffusion.

• Proposed model without MC dropout.

• Proposed model without aleatoric variance, if this ablation is well-defined.

• Persistence/last-observation baseline.

• Relevant published baselines, labelled as reproduced or literature-reported.

Fair comparison: Published values are not matched experiments unless the baseline uses the same split, preprocessing and metric definitions. Label each result accordingly.

8. Add realistic sensor-failure experiments

Priority: Strongly recommended
Why: Independent random dropout alone does not cover practical failure patterns.

Files to inspect: Robustness evaluation script and corruption-generation utilities.

1. Evaluate clean input, random sensor dropout, persistent sensor failure, spatially clustered failure and temporally contiguous missing blocks.

2. Start with 10%, 20% and 30% corruption rates if feasible. Define whether the rate applies to nodes, sensor-time observations or whole windows.

3. Apply the same corrupted input to every compared model; keep targets and evaluation masks consistent.

4. Report MAE/RMSE and degradation relative to clean input; repeat across corruption seeds.

5. Separate original missingness from artificial test-time corruption and include a simple imputation baseline if missing-input handling is claimed.

Deliverables: Tables and plots of error versus corruption type/rate with repeated-seed summaries.

9. Additional metrics that can strengthen the paper

Metric / analysis	What to report	Why it matters
Forecast accuracy Essential	MAE and RMSE per horizon and dataset; mean ± SD across seeds.	Shows degradation as forecast horizon increases.
Calibration Essential	Coverage at 50%, 80%, 90%, 95%; mean interval width; nominal-vs-empirical plot.	Checks reliability and whether intervals are excessively wide.
Probabilistic quality Strongly recommended	CRPS when a defensible predictive distribution is available; otherwise interval score or weighted interval score.	Proper scores evaluate both accuracy and uncertainty; coverage alone is insufficient.
Robustness Strongly recommended	Error degradation versus clean input for random, persistent, spatial and temporal-block failures.	Quantifies performance under realistic sensor failures.
Efficiency Recommended	Parameter count, inference latency, peak memory, throughput; state hardware, batch size and method.	Supports deployment claims better than parameter count alone.
Statistical reliability Recommended	Confidence intervals across runs and paired comparisons on same test windows where appropriate; account for multiple comparisons.	Helps judge whether differences are robust to randomness.

Metric caution: Avoid MAPE as a headline metric without justification because zero or near-zero speeds can make percentage errors unstable. Do not let one ECE number replace coverage plots and interval-width measures.

10. Suggested division of work between two students

Student 1 — Data pipeline and model correctness

• Fix chronological splitting, train-only scaler/imputer fitting and causal missing-value handling.

• Verify tensor shapes and correct temporal convolution/LSTM axes if needed.

• Audit graph construction and diffusion direction.

• Correct dropout/batch-normalisation behaviour during MC sampling.

• Add tests for split boundaries, tensor dimensions and uncertainty output shapes.

• **Deliver:** corrected code, change log, passing tests and one documented smoke-test run.

Student 2 — Evaluation, calibration and experiments

• Correct uncertainty aggregation and prediction-interval calculations.

• Use validation-only checkpoint selection and prevent repeated test-set inspection.

• Implement metric calculations and horizon-wise reporting.

• Prepare repeated-seed runs and diffusion/uncertainty ablations.

• Prepare clean-input and sensor-failure evaluations.

• **Deliver:** evaluation scripts, saved predictions, metric tables, calibration plots and experiment logs.

Joint responsibility: Agree on tensor conventions, split boundaries and metric definitions; review each other's changes; run final experiments from the same corrected codebase; make every table and figure reproducible from saved outputs.

Dependency: If architecture changes affect outputs, Student 2 should wait until Student 1's implementation passes shape and correctness tests.

11. Final checklist before accepting results

Data and model correctness

■ Preprocessing fitted on training data only

■ No future-based imputation in forecasting inputs

■ Chronological split and window boundaries verified

■ Temporal operations act on the time axis

■ Graph orientation and diffusion verified

- MC dropout tested without updating batch-normalisation statistics

Evaluation

- Validation-only checkpoint selection
- Test set evaluated only after decisions are final
- Aleatoric and epistemic variance combined correctly
- Prediction intervals and calibration metrics verified
- Diffusion ablation reproduced and checked
- At least five seeds for final core results, or limitation documented

Experimental evidence

- Results separated by dataset and horizon
- Clean and sensor-failure results saved
- Baselines labelled as reproduced or literature-reported
- Mean and standard deviation reported
- Inference latency and memory measured if deployment is claimed

Paper consistency

- Tables and plots regenerated from corrected outputs
- All manuscript numbers updated
- Architecture diagram and equations match code
- Limitations and experimental protocol documented

Recommended order of work

1. Correct data preprocessing and architecture.

2. Verify graph operations and uncertainty calculations.

3. Run one end-to-end experiment and inspect outputs.

4. Run multi-seed experiments, ablations and robustness tests.

5. Regenerate paper tables and figures from corrected outputs.

Final recommendation

Do not spend the full experimental budget repeating the current implementation before checking correctness. If correcting the temporal architecture materially changes the model, the core results, methodology and conclusions may need to change. Report corrected evidence transparently, even if it is less favourable.

Scope note: This is a practical plan based on the preceding source-code review. Suspected issues should be confirmed against actual code and tests before being treated as established implementation failures. This plan does not claim that experiments have already been rerun.